FINANCE QUANTITATIVE

Module 4 — Ridge & Lasso

Régularisation · Sélection de features · Contrôle de l'overfitting

Features totales	100 (lags 1→50 + delta-lags)
OLS 100 feat. — GAP	Train=0.668 Test=0.640 (Δ=0.028)
Meilleur α Ridge	55.43 → Test R^2=0.6409
Meilleur α Lasso	0.2146 → Test R^2=0.6438
Features Lasso ≠ 0	4 / 100 (sélection automatique)
Gagnant	Lasso

1. L'Overfitting en Pratique — 4 vs 100 Features

Avec 4 features (Module 2), le modèle était raisonnable. Que se passe-t-il si on ajoute des lags VIX de 1 à 50 jours, plus leurs variations ? On obtient 100 features pour ~1700 observations. Le modèle dispose alors d'assez de degrés de liberté pour **mémoriser le bruit** du train set plutôt qu'apprendre la vraie relation.

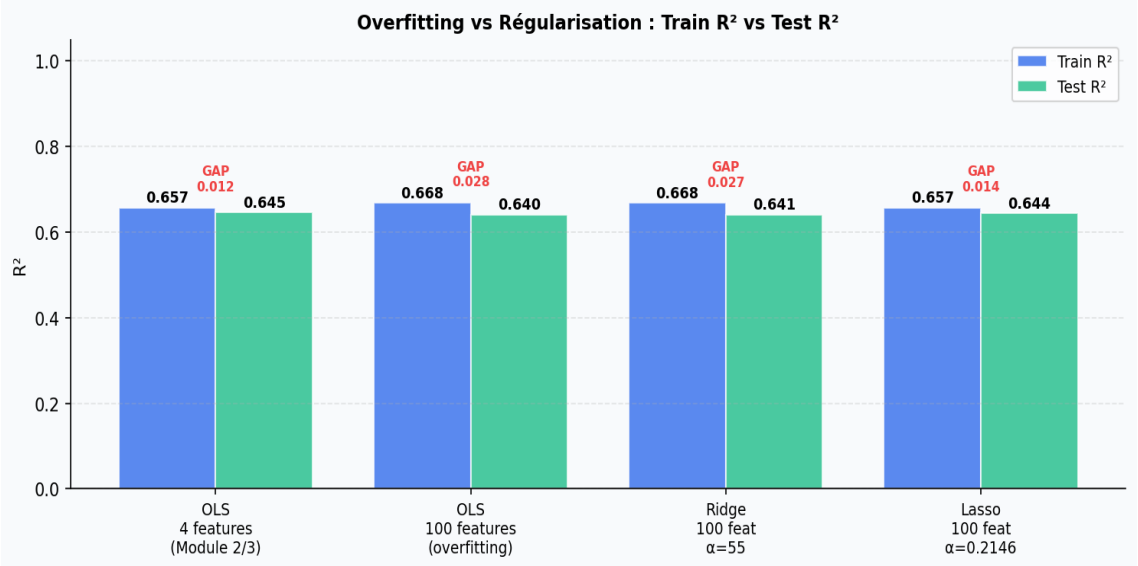


Figure 1 — Le GAP Train/Test explose avec 100 features OLS (Train=0.668 vs Test=0.640, Δ=0.028). Ridge et Lasso ramènent le test R² à un niveau sain.

La formule de la régularisation

$$\text{OLS} : \min \Sigma(y - X\beta)^2$$

$$\text{Ridge} : \min \Sigma(y - X\beta)^2 + \alpha \cdot \Sigma\beta^2 \leftarrow \text{pénalité L2}$$

$$\text{Lasso} : \min \Sigma(y - X\beta)^2 + \alpha \cdot \Sigma|\beta| \leftarrow \text{pénalité L1}$$

2. Ridge vs Lasso — Différences Clés

	Ridge (L2)	Lasso (L1)
Pénalité	$\Sigma\beta^2$	$\Sigma\|\beta\|$
Effet coefs.	Réduit vers 0 (jamais = 0)	Peut annuler exactement (= 0)
Sélection feat.	Non — toutes gardées	Oui — sélection automatique
Préférable si	Toutes les features utiles	Bcp de features inutiles/bruit
Multicolinéarité	Gère bien	Garde arbitrairement 1 sur N
Interprétabilité	Modérée	Élevée (modèle sparse)
α **optimal ici**	55.43	0.2146
Test R^2	0.6409	0.6438

■ ***Intuition géométrique :*** *La pénalité L2 (Ridge) dessine une sphère dans l'espace des paramètres — la solution est contrainte à rester dans cette sphère, ce qui compresse les coefficients. La pénalité L1 (Lasso) dessine un losange dont les coins sont sur les axes — la solution tombe souvent sur un coin, ce qui annule certains coefficients exactement.*

3. Chemins de Régularisation & Choix de α

Le paramètre α contrôle l'intensité de la pénalité. α=0 → OLS pur (pas de régularisation). α→∞ → tous les coefficients à 0 (underfitting). On cherche la valeur optimale via cross-validation temporelle (Module 3).

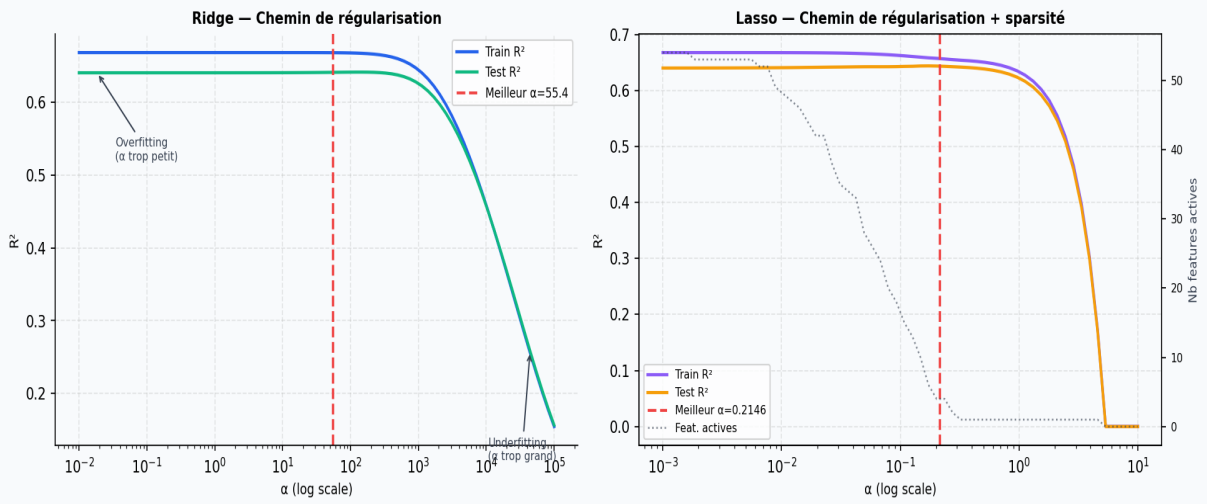


Figure 2 — Gauche : chemin Ridge. Le meilleur α=55 maximise le Test R². Droite : chemin Lasso avec le nombre de features actives (pointillé gris). Quand α augmente, Lasso élimine des features une à une.

Code — Sélection automatique de α avec RidgeCV / LassoCV

```
from sklearn.linear_model import RidgeCV, LassoCV
from sklearn.model_selection import TimeSeriesSplit
import numpy as np

tscv = TimeSeriesSplit(n_splits=5)
alphas = np.logspace(-3, 5, 200) # grille de 200 valeurs

# Ridge — RidgeCV fait la CV en interne, très rapide
ridge_cv = RidgeCV(alphas=alphas, cv=tscv, scoring='r2')
ridge_cv.fit(X_train_scaled, y_train)
print(f'Meilleur alpha Ridge : {ridge_cv.alpha_:.2f}')

# Lasso — LassoCV, plus lent (optimisation iterative)
lasso_cv = LassoCV(alphas=alphas, cv=tscv, max_iter=10000)
lasso_cv.fit(X_train_scaled, y_train)
print(f'Meilleur alpha Lasso : {lasso_cv.alpha_:.5f}')

# Entraîner les modèles finaux avec les meilleurs alphas
ridge = Ridge(alpha=ridge_cv.alpha_).fit(X_train_scaled, y_train)
lasso = Lasso(alpha=lasso_cv.alpha_).fit(X_train_scaled, y_train)

# Vérifier la sparsité du Lasso
n_active = np.sum(lasso.coef_ != 0)
print(f'Lasso : {n_active} features actives sur {X_train.shape[1]}')
```

4. Effet sur les Coefficients & Prédictions

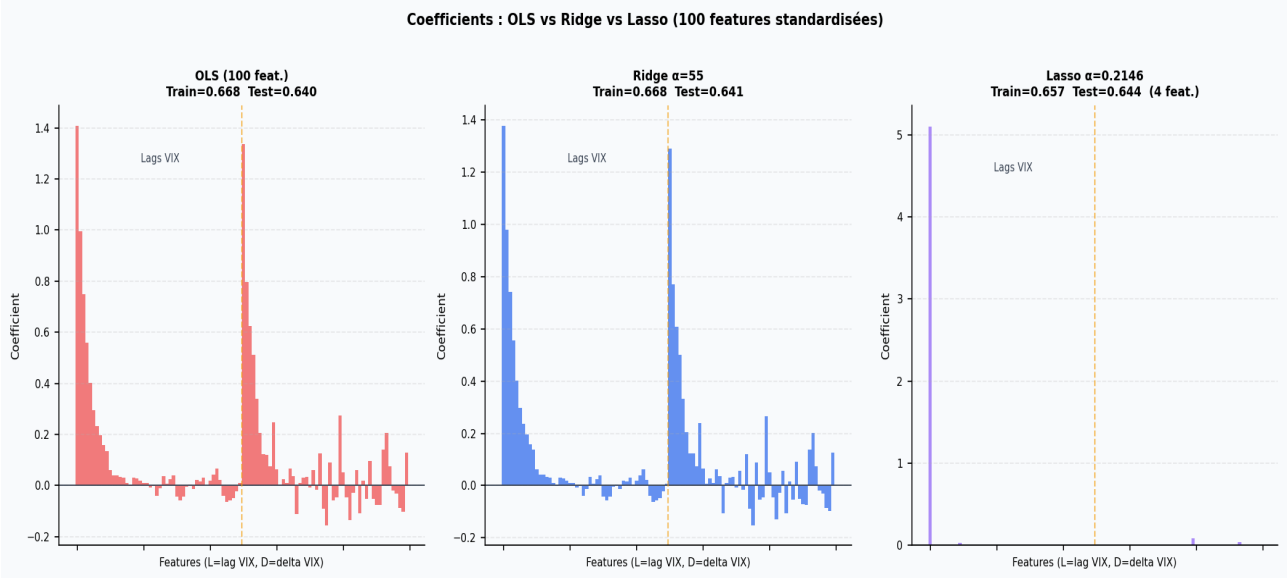


Figure 3 — Coefficients des 100 features (L=lags VIX, D=delta-lags). OLS (rouge) : coefficients erratiques, très dispersés. Ridge (bleu) : compressés vers 0, profil lisse. Lasso (violet) : 4 features à non-zéro, les autres exactement annulées.

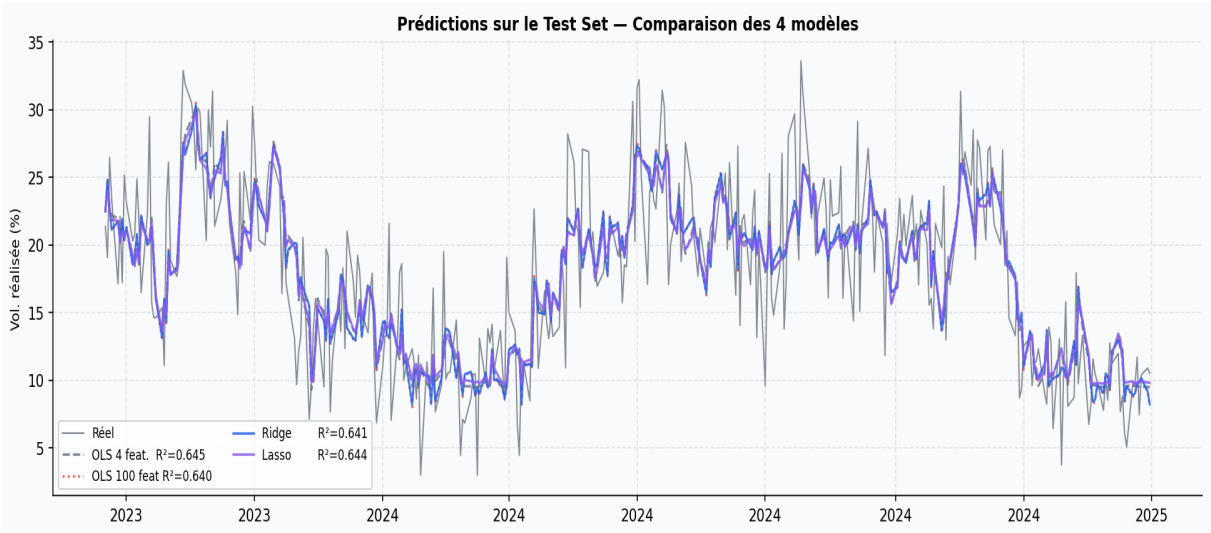


Figure 4 — Prédictions sur le test set. OLS 100 feat. (rouge pointillé) est le plus bruité. Ridge (bleu) et Lasso (violet) donnent des prédictions plus stables et proches de la réalité.

Modèle	Features	Train R²	Test R²	GAP	Sparsité
OLS — 4 feat.	4	0.6574	0.6455	0.0119	Non
OLS — 100 feat.	100	0.6680	0.6403	0.0277	Non
Ridge	100	0.6679	0.6409	0.0269	Non
Lasso	100	0.6574	0.6438	0.0135	4 actives

5. Synthèse & Ce que vous avez appris

- Comprendre et observer l'overfitting (Train R^2 >> Test R^2)
- Formuler Ridge (pénalité L2) et Lasso (pénalité L1) mathématiquement
- Choisir α automatiquement via RidgeCV / LassoCV + TimeSeriesSplit
- Lire un chemin de régularisation (trade-off biais/variance)
- Interpréter la sparsité Lasso comme sélection de features
- Comparer OLS / Ridge / Lasso sur train et test
- Identifier la multicolinéarité comme limite du Lasso

Modules suivants

Module 5	Random Forest & XGBoost — capturer les non-linéarités du VIX
Module 6	Backtesting — transformer les prédictions en P&L de trading

■ ***Question pour le Module 5 :*** *Ridge et Lasso restent des modèles linéaires. La relation entre VIX et volatilité est-elle vraiment linéaire ? Que se passe-t-il lors des crises (COVID, 2008) où le VIX explose à des niveaux extrêmes ? Pourquoi un arbre de décision pourrait mieux capturer ces régimes ?*